Software Requirements Specification

for

WalkingSG

Version 10.0

Prepared by Trinh Hoang Huy

Group 3 – SCS2

September 20th

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Table of Contents

Table of Contents ..... **ii**
Revision History ..... **iiii**
1. Introduction **1**
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 1
1.4 Product Scope 1
1.5 References 1
2. Overall Description **2**
2.1 Product Perspective 2
2.2 Product Functions 2
2.3 User Classes and Characteristics 2
2.4 Operating Environment 2
2.5 Design and Implementation Constraints 2
2.6 User Documentation 2
2.7 Assumptions and Dependencies 3
3. External Interface Requirements **4**
3.1 User Interfaces 4
3.2 Hardware Interfaces 9
3.3 Software Interfaces 9
3.4 Communications Interfaces 9
4. System Features **9**
4.1 Authentication 9
4.2 Preference Management 11
4.3 Search & Map Interface 12
4.4 Routing & Scoring Engine 13
4.5 Live Navigation 14
4.6 Community Hazard & Local Feed 15
5. Other Nonfunctional Requirements **16**
5.1 Performance Requirements 16
5.2 Safety Requirements 16
5.3 Security Requirements 17
5.4 Software Quality Attributes 17
5.5 Business Rules 17
6. Other Requirements **19**
Appendix A: Analysis Models **19**
Appendix B: To Be Determined List **23**
Appendix C: Data Dictionary **24**

Revision History

Name	Date	Reason For Changes	Version
Trinh Hoang Huy	August 31st	Initial Requirements Proposal	1.0
Hemanth	September 4th	Update Data Dictionary	2.0
Trinh Hoang Huy	September 5th	Refine Functional Requirements	3.0
Lin Ziqi	September 5th	Update UI Update Content page	4.0
Trinh Hoang Huy	September 5th	Final touch on the first SRS Version (Reformat for better look)	5.0
Fujita Natsuo	September 16th	Refined UI	6.0
Trinh Hoang Huy	September 17th	Fill in the remaining placeholders	7.0
Lin Ziqi	September 18th	Added in class diagram of entity class	8.0
Fujita Natsuo	September 18th	Added in initial dialog diagram	9.0
Trinh Hoang Huy	September 20th	Added Boundary & Control Classes	10.0

1. Introduction

1.1 Purpose

- This system is a pedestrian-first, mobile-based navigation application scoped specifically to the geographic boundaries of Singapore. It provides personalized pedestrian routing based on user mobility constraints, real-time weather, and community-driven accessibility scoring.

1.2 Document Conventions

- Functional requirements are identified by a unique sequence tag (e.g., REQ-AUTH-01). Priorities (High, Medium, Low) are explicitly stated for major features and inherited by their sub-requirements.

1.3 Intended Audience and Reading Suggestions

- This document is intended for backend developers, frontend mobile developers, UI/UX designers, and project managers. Developers should focus heavily on Section 4 (System Features) and Section 5 (Other Nonfunctional Requirements), while designers should focus on Section 3 (External Interface Requirements).

1.4 Product Scope

- WalkingSG aims to solve the "last mile" and pedestrian navigation gaps left by vehicle-centric apps. The software allows users to define physical mobility needs (e.g., wheelchair) and environmental preferences (e.g., maximum shelter). It calculates five distinct routes for every query, ranking the top choice dynamically based on the user's profile. It relies on community reports to adjust the "confidence score" of individual map segments, ensuring real-time hazard avoidance.

1.5 References

- Singapore Land Authority (SLA) OneMap API Documentation (v2.0 RESTful Services: Search, Reverse Geocoding, Barrier-Free Pedestrian Routing).
- Singapore Active Mobility Act 2017 (Cap. 2C) (Pedestrian and mobility aid regulations).
- Singapore Land Transport Authority (LTA) Walk2Ride Programme & Code of Practice for Street Works (Sheltered Walkways & Accessibility Guidelines).
- National Environment Agency (NEA) & Meteorological Service Singapore (MSS) Weather and UV Index Data APIs.
- React Native & Expo Documentation (Expo SDK 52+, React Navigation v6, React Native 0.86).

2. Overall Description

2.1 Product Perspective

- WalkingSG is a new, self-contained mobile application. It operates as a front-end interface that relies heavily on external mapping APIs (OneMap, OpenStreetMap) for baseline geospatial data, while maintaining its own proprietary database for user profiles, saved preferences, and custom map-segment confidence scores.

2.2 Product Functions

- Account creation and password recovery.
- Management of personal mobility and environmental preferences.
- Calculation and display of 5 distinct pedestrian route types (Direct, Maximum Shelter, Barrier-Free, Low-Effort, Wide/Quiet).
- Live, GPS-based turn-by-turn navigation.
- Community hazard reporting and dynamic path-segment scoring.
- Location-based Local Hazard News Feed.

2.3 User Classes and Characteristics

- **Standard Pedestrians:** High frequency of use, seeking sheltered or direct routes.
- **Mobility-Impaired Users (Wheelchair):** Require high accuracy on step-free ("Barrier-Free") routing. Zero tolerance for unmapped stairs.
- **Elderly Walkers:** Prefer "Low-Effort" routes avoiding steep inclines.

2.4 Operating Environment

- The software will operate as a cross-platform mobile application (iOS 15+ and Android 10+). It requires an active internet connection (4G/5G/WiFi) and active device GPS functionality.

2.5 Design and Implementation Constraints

- Geographic scope is strictly locked to Singapore's coordinate bounding box.
- The system shall not calculate vehicular, bicycle, or public transit routing. It is exclusively optimized for walking and pedestrian-supportive personal mobility aids.
- Data consumption must be minimized to maintain performance on typical mobile data plans.

2.6 User Documentation

- **Interactive In-App Onboarding Flow**: First-launch tutorial explaining GPS permissions, Singapore bounding geofence, and Personal Declaration setup.

- **WalkingSG User Manual (PDF / Markdown)**: End-user instructions for route variant selection (Direct, Sheltered, Barrier-Free, Low-Effort, Wide/Quiet), live navigation, and community reports.

2.7 Assumptions and Dependencies

2.7.1 Assumptions

- **GPS Signal Availability:** It is assumed that the end-user's device has a functional GPS sensor capable of maintaining an accuracy threshold of within 10 meters, and that Singapore's urban canyons (e.g., the Central Business District or Marina Bay) do not consistently block satellite positioning signals.
- **Continuous Cellular Internet:** It is assumed that users maintain active mobile data packages (4G/5G) or are connected to urban public WiFi networks during routing queries and live navigation.
- **Active Mobility Compliance:** It is assumed that standard pedestrians and mobility-impaired users comply with physical path markers, speed regulations under the Singapore Active Mobility Act, and real-world construction signs, treating app recommendations as advisory.

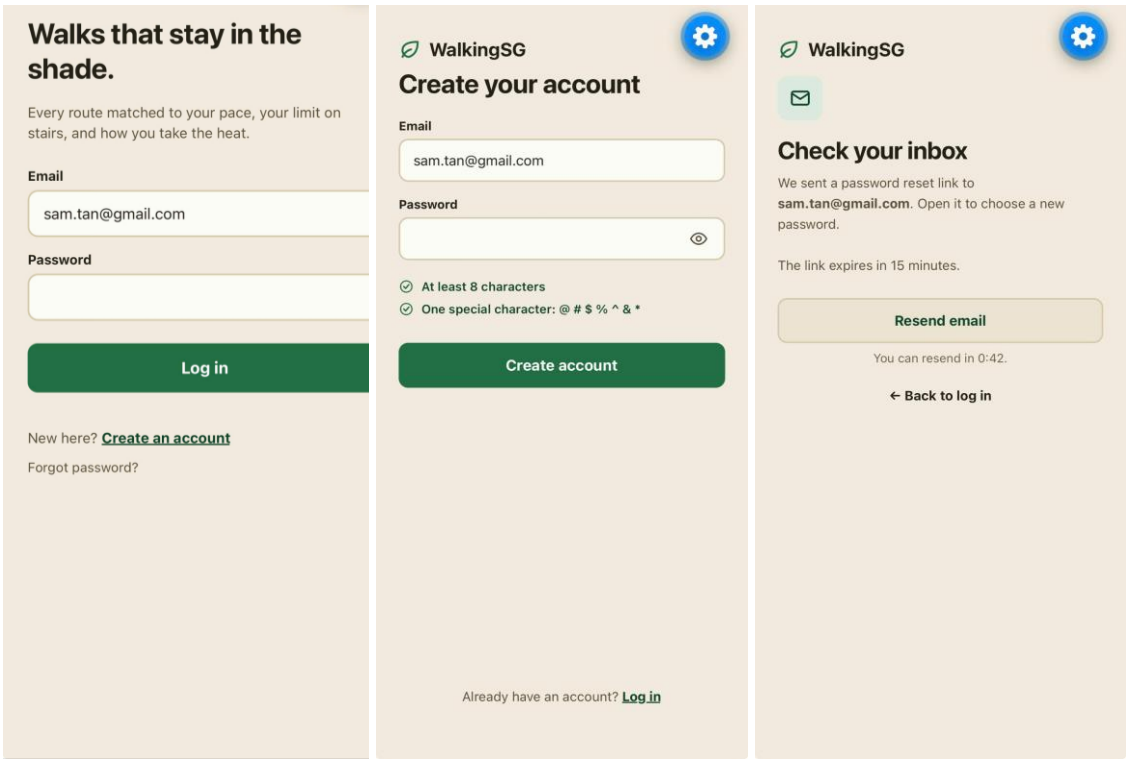
2.7.2 Dependencies

- **Singapore Land Authority (SLA) OneMap APIs:** The system depends entirely on SLA's OneMap APIs for baseline address lookup, autocomplete suggestions, base map tiles, and foundational wheelchair-accessible networks. Any service disruption on SLA's production server directly degrades routing capabilities.
- **Firebase Cloud Messaging (FCM) & Apple Push Notifications (APNs):** The instant local hazard news broadcasting mechanism relies on steady integrations with third-party cloud messaging push pipelines to reach active devices within 5-kilometer bounding boxes.

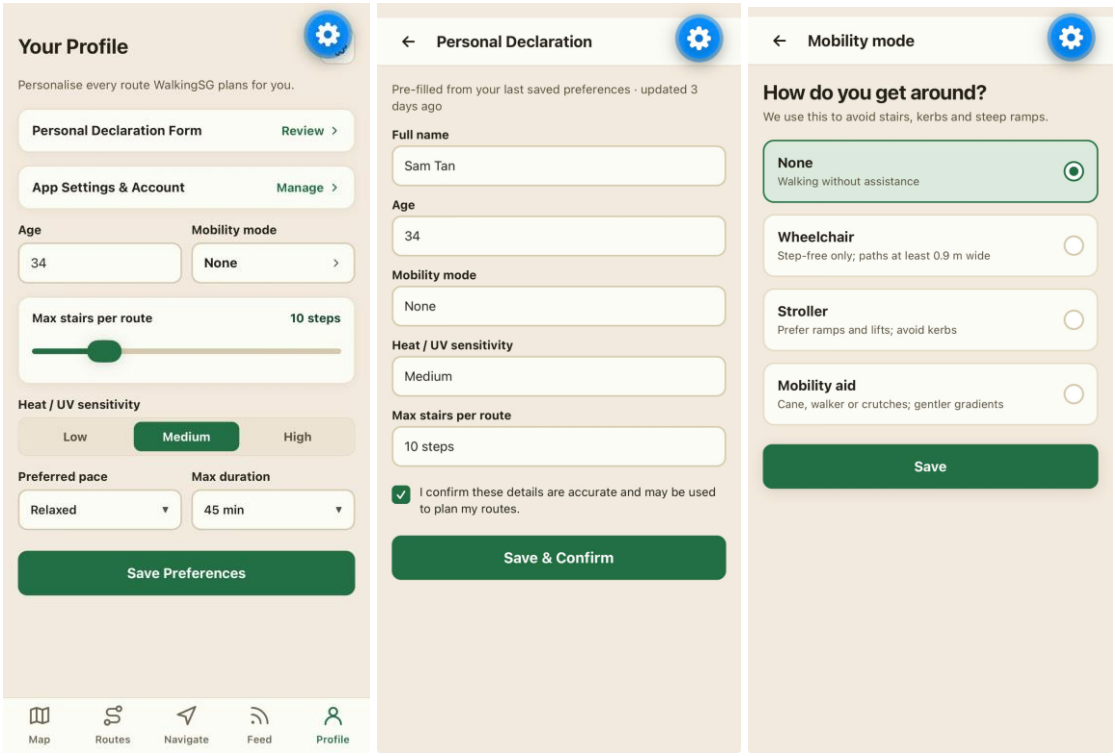
3. External Interface Requirements

3.1 User Interfaces

Login Screen



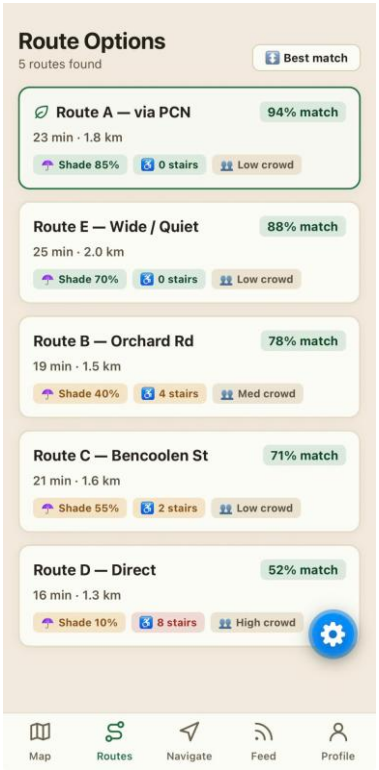
Profile and Preferences Screens



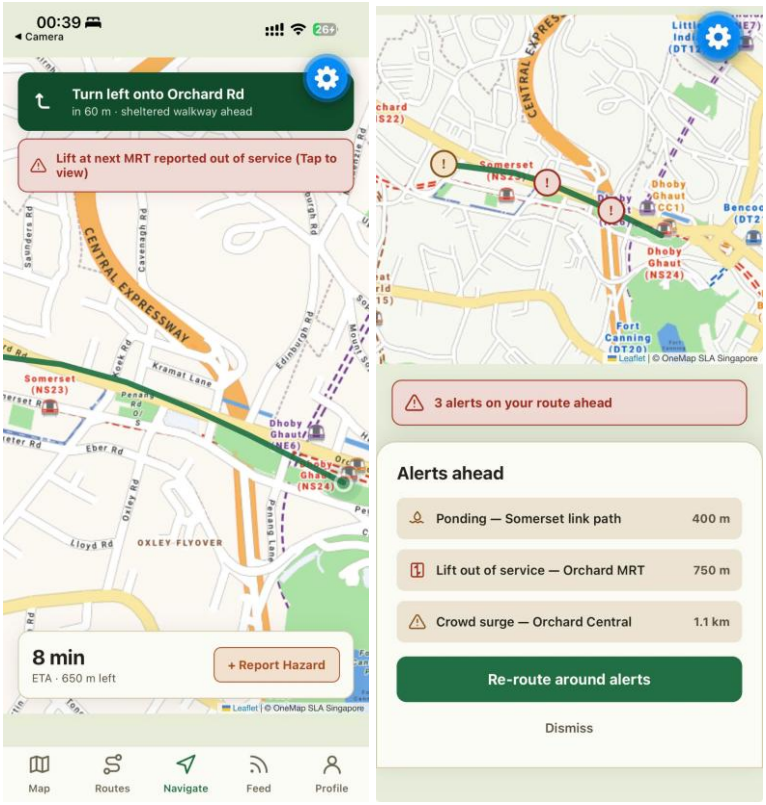
Search and Map Screen



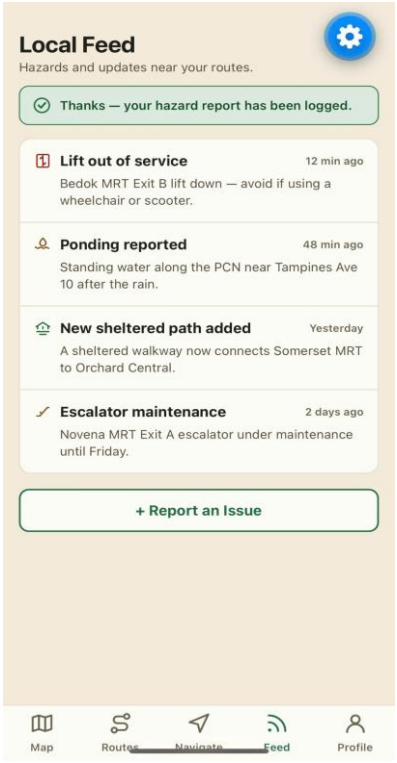
Route Result Screen

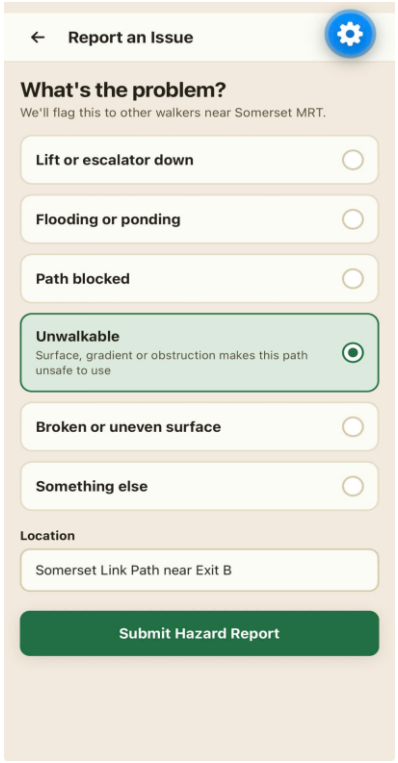


Live Navigation Screen

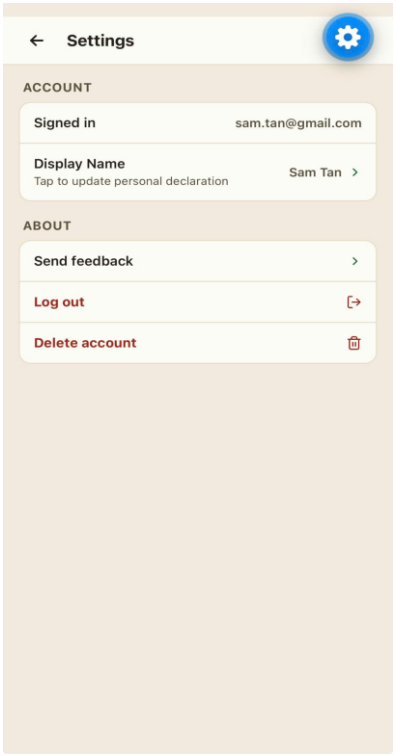


Local Feed Screen





Setting Screen



3.2 Hardware Interfaces

- The software must interface with the mobile device's internal GPS sensor to receive continuous coordinate updates (latitude/longitude/heading) during active navigation.

3.3 Software Interfaces

- **OneMap API:** Used for address autocomplete, base map rendering, and baseline barrier-free routing data.
- **Database (e.g., PostgreSQL):** Stores user profiles, hashed passwords, and path-segment confidence tables.

3.4 Communications Interfaces

- **Client-Server Protocol:** HTTPS for all REST API endpoints**.**
- **Mapping & Geolocation Services:** External SLA OneMap HTTPS REST API for address search, geocoding, and polyline coordinates.
- **Transactional Email Dispatch:** Secure SMTP / RESTful email service (TLS encrypted) for 6-digit registration OTPs and password reset tokens.
- **Network Resiliency:** 5.0s client timeout threshold with exponential backoff retry mechanism (max 3 attempts) for cellular connectivity.

4. System Features

4.1 Authentication

4.1.1 Description and Priority

- **High priority**. Handles secure user access, account creation, credential validation, and session lifecycle management.

4.1.2 Stimulus/Response Sequences

- User opens app -> System prompts login/register -> User submits credentials -> System validates against database -> System authenticates, generates session token, and grants access.

4.1.3 Functional Requirements

REQ-AUTH-01: The system must allow a user to register a new account using an email address and password.

- **REQ-AUTH-01.1:** The system must verify that the provided email address is not already linked to an existing account.
- **REQ-AUTH-01.2:** The system must display the error message "Email already in use" if a duplicate email registration is attempted.
- **REQ-AUTH-01.3:** The system must enforce a minimum password length of 8 characters.

- **REQ-AUTH-01.4:** The system must enforce the inclusion of at least one special character (e.g., @, #, $, %, ^, &, *) in the password.
- **REQ-AUTH-01.5:** The system must verify the ownership of the email address with a one-time 6-digit code sent to the registered email.
- **REQ-AUTH-01.6:** The system must expire the one-time code 10 minutes after it is issued.
- **REQ-AUTH-01.7:** The system must activate the account only after successful code verification.

REQ-AUTH-02: The system must allow a user to log in using their registered email and password.

- **REQ-AUTH-02.1:** The system must deny access and display the error message "Invalid email or password" if the provided credentials do not match database records.
- **REQ-AUTH-02.2:** The system must lock the user account for 15 minutes after 5 consecutive failed login attempts.

REQ-AUTH-03: The system must provide a "Forgot Password" function to recover lost account access.

- **REQ-AUTH-03.1:** The system must send a password reset link to the user's registered email address upon request.
- **REQ-AUTH-03.2:** The system must invalidate the password reset link 15 minutes after it is generated.
- **REQ-AUTH-03.3:** The system must enforce the standard password strength requirements (**REQ-AUTH-01.3**, **REQ-AUTH-01.4**) when the user sets a new password.

REQ-AUTH-04: The system must allow an authenticated user to securely log out.

- **REQ-AUTH-04.1:** The system must terminate the active session token upon logout and redirect the user to the login screen.

REQ-AUTH-05: The system must allow users to edit and manage their profile.

- **REQ-AUTH-05.1:** The system must allow users to update their displayed Full Name, validating that the input is non-empty and between 2 and 50 characters.
- **REQ-AUTH-05.2:** The system must allow users to delete their account
- **REQ-AUTH-05.3:** The system must prompt the user for confirmation before account deletion.
- **REQ-AUTH-05.4:** The system must permanently delete the user's account and associated data from the database once confirmed.

4.2 Preference Management

4.2.1 Description and Priority

- **High priority**. Manages the user's physical mobility constraints and environmental preferences that dynamically dictate the routing engine's weightage formula.

4.2.2 Stimulus/Response Sequences

- User navigates to profile -> User modifies mobility or route preferences -> System validates inputs -> System saves preferences to database -> System applies new weights to all subsequent route calculations.

4.2.3 Functional Requirements

REQ-PREF-01: The system must display a "Personal Declaration Form" within the user profile interface.

- **REQ-PREF-01.1:** The system must automatically pre-fill the form with the user's most recently saved database preferences upon loading.

REQ-PREF-02: The system must provide a dropdown menu for “mobility_needs” to dictate physical routing constraints.

- **REQ-PREF-02.1:** The “mobility_needs” dropdown must strictly contain the following options: None, Wheelchair, Mobility Aid.
- **REQ-PREF-02.2:** The system must default the “mobility_needs” field to "None" for newly registered accounts.

REQ-PREF-03: The system must allow the user to save their modified preferences.

- **REQ-PREF-03.1:** The system must commit the updated form data to the database under the active user's unique ID.
- **REQ-PREF-03.2:** The system must immediately apply the updated variables to the routing engine for the very next search query.

REQ-PREF-04: The system must provide a numeric input field for "Age" within the user profile interface.

- **REQ-PREF-04.1:** The system must only accept positive integers for the age field.

REQ-PREF-05: The system must provide a selection for "Heat / UV sensitivity" to dictate routing preferences regarding shaded or indoor paths.

- **REQ-PREF-05.1:** The "Heat / UV sensitivity" selection must strictly contain the following options: Low, Medium, High.
- **REQ-PREF-05.2:** The system must default this field to "Medium" for newly registered accounts.

REQ-PREF-06: The system must provide a slider input for "Max stairs per route" accepting values from 0 to 50 steps in increments of 5, defaulting to 10 steps for

standard users and locked to 0 steps if “mobility_needs” is "Wheelchair".

REQ-PREF-07: The system must provide a dropdown menu for "Preferred pace" with options: "Slow" (3.0 km/h), "Relaxed" (4.0 km/h), and "Brisk" (5.5 km/h), defaulting to "Relaxed".

4.3 Search & Map Interface

4.3.1 Description and Priority

- **High priority**. Handles the primary user interface where the map is rendered, and origin/destination points are defined.

4.3.2 Stimulus/Response Sequences

- User grants GPS permission → System renders map and centers on current location → User inputs destination → System validates the coordinates.

4.3.3 Functional Requirements

REQ-MAP-01: The system must render an interactive map interface using the selected external map provider API.

- **REQ-MAP-01.1:** The system must allow the user to interactively zoom and pan the map interface.

REQ-MAP-02: The system must prompt the user for device location (GPS) permissions upon first entering the main UI.

- **REQ-MAP-02.1:** The system must auto-fill the "Origin" input field with the user's current GPS location if permission is granted.
- **REQ-MAP-02.2:** The system must leave the "Origin" input field blank for manual entry if GPS permission is denied.

REQ-MAP-03: The system must provide autocomplete suggestions for address searches using the external Search API.

REQ-MAP-04: The system must validate all input Origin and Destination coordinates against the Singapore geographical bounding box.

- **REQ-MAP-04.1:** The system must reject the routing request if the Origin coordinate falls outside the predefined Singapore bounding box.
- **REQ-MAP-04.2:** The system must display the text "Invalid location. Please select a valid origin within Singapore" if the Origin coordinate falls outside the predefined Singapore bounding box.
- **REQ-MAP-04.3:** The system must reject the routing request if the Destination coordinate falls outside the predefined Singapore bounding box.
- **REQ-MAP-04.4:** The system must display the text "Invalid location. Please select a valid destination within Singapore" if the Destination coordinate falls outside the predefined Singapore bounding box.

REQ-MAP-05: The system must display markers for the origin and destination on the map interface once selected.

4.4 Routing & Scoring Engine

4.4.1 Description and Priority

- **High priority**. The core algorithmic backend that generates multiple paths, applies the user's preferences, and ranks the results based on segment confidence.

4.4.2 Stimulus/Response Sequences

- System receives valid Origin/Destination → Engine calculates 5 routes → Engine applies profile weights → System displays sorted routes.

4.4.3 Functional Requirements

REQ-RTE-01: The routing engine must attempt to generate up to 5 distinct route variants for every valid search query.

- **REQ-RTE-01.1:** If the user's "mobility_needs" is set to "Wheelchair", the engine must strictly exclude all segments containing steps, escalators, or vertical curbs from all route variants before applying preference optimizations.
- **REQ-RTE-01.2:** The engine must generate a "Direct Route" optimized solely for the shortest physical distance.
- **REQ-RTE-01.3**: The engine must generate a "Maximum Shelter Route" maximizing covered linkways and indoor paths.
- **REQ-RTE-01.4:** The engine must generate a "Barrier-Free Route" containing zero stairs, escalators, or curbs.
- **REQ-RTE-01.5:** The engine must generate a "Low-Effort Route" minimizing steep elevation changes.
- **REQ-RTE-01.6:** The engine must generate a "Wide/Quiet Route" maximizing Park Connector Network (PCN) usage.
- **REQ-RTE-01.7:** The system must return a "No route available" payload for a specific route variant if no path satisfies that variant's unique physical constraints, while continuing to generate the remaining viable variants.

REQ-RTE-02: The system must process routing metadata for each generated route.

- **REQ-RTE-02.1:** The routing engine must calculate the total estimated walking time.
- **REQ-RTE-02.2:** The routing engine must calculate the total physical distance.

- **REQ-RTE-02.3:** The routing engine must calculate the sheltered distance percentage.
- **REQ-RTE-02.4:** The system must append the calculated walking time to the route response payload.
- **REQ-RTE-02.5:** The system must append the calculated distance to the route response payload.
- **REQ-RTE-02.6:** The system must append the calculated sheltered percentage to the route response payload.
- **REQ-RTE-02.7:** The system must display "Distance information not available" or "Estimated travel time not available" if the respective data is incomplete.

REQ-RTE-03: The routing engine must filter out any map segment possessing a confidence score less than -10 when generating the "Barrier-Free Route".

REQ-RTE-04: The system must calculate a final match score on a scale of 0 to 100 for each route using the user's saved mobility profile (“mobility_needs”, “age”, “heat_uv_sensitivity”, “max_stairs_per_route”, and “preferred_pace”).

REQ-RTE-05: The system must present the 5 routes to the user in a UI drawer.

- **REQ-RTE-05.1:** The system must sort the displayed routes in descending order based on their match score.
  - **REQ-RTE-05.1.1:** The system must resolve identical match scores by prioritizing the route with the shorter estimated walking time.
  - **REQ-RTE-05.1.2:** The system must prioritize the route with the higher sheltered distance percentage if the estimated walking times are also tied.
- **REQ-RTE-05.2:** The system must visually highlight the route with the highest match score with a "Starred" badge.

4.5 Live Navigation

4.5.1 Description and Priority

- **High priority**. Provides the active turn-by-turn guidance once a user selects a route.

4.5.2 Stimulus/Response Sequences

- User selects a route → System initiates tracking → User moves → System updates instructions → User reaches destination → System updates segment confidence.

4.5.3 Functional Requirements

REQ-NAV-01: The system must anchor the map view to the user's real-time GPS coordinate during active navigation.

REQ-NAV-02: The system must display a text-based directional instruction.

- **REQ-NAV-02.1:** The system must dynamically update the text instruction based on the user's progression along the route polyline.

REQ-NAV-03: The system must monitor the user's deviation from the active route.

- **REQ-NAV-03.1:** The system must trigger an automatic route recalculation for current active route type if the user's GPS coordinate deviates from the active path by more than 20 meters for 5 consecutive seconds.
- **REQ-NAV-03.2:** The system must take the current location of the user as the new origin during recalculation.

REQ-NAV-04: The system must detect when the user arrives at the destination.

- **REQ-NAV-04.1:** The system must consider the route completed when the user's GPS coordinate is within a 5-meter radius of the destination point.
- **REQ-NAV-04.2:** The system must automatically increment the confidence score of all traversed map segments by +1 upon successful route completion, up to a maximum clamp limit of +20.

REQ-NAV-05: The system must automatically update the metric information (remaining distance and estimated time) as the user travels through the route.

4.6 Community Hazard & Local Feed

4.6.1 Description and Priority

- **Medium priority**. Handles the crowdsourced data loop, allowing users to report unwalkable checkpoints and view local warnings.

4.6.2 Stimulus/Response Sequences

- User taps “Live Report” → System logs hazard → System updates database → System displays hazard on the local feed for nearby users.

4.6.3 Functional Requirements

REQ-HAZ-01: The system must display a "Live Report" button on the screen during active navigation.

REQ-HAZ-02: The system must allow the user to report a map segment as "Unwalkable", “Path blocked”, “Flooding”, “Something Else”, “Lift down” and “Broken Surface” via the Live Report button.

- **REQ-HAZ-02.1:** The system must prompt the user for confirmation before submitting the report.
- **REQ-HAZ-02.2:** The system must map the hazard to the specific route segment closest to the user's current GPS coordinate.

- **REQ-HAZ-02.3:** The system must deduct 2 points from that specific map segment's confidence score in the database upon report confirmation, down to a minimum clamp limit of -20.

REQ-HAZ-03: The system must provide a dedicated "News/Hazards" screen.

- **REQ-HAZ-03.1:** The system must query the database for all map segments have received a confirmed hazard report within the past 72 hours, or with a negative confidence score.
- **REQ-HAZ-03.2:** The system must filter the queried hazards to only include those located within a strictly 5-kilometer radius of the user's current GPS location.
- **REQ-HAZ-03.3:** The system must display the filtered list of hazards to the user.
- **REQ-HAZ-03.4:** The system must sort the displayed hazard list chronologically, showing the most recently reported hazards at the top.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- The system must render the 5 calculated route variants on the map interface within 3.0 seconds of the user submitting a valid destination over a standard 4G connection.
- The system must display address autocomplete suggestions within 500 milliseconds of user keystroke input during search.
- The system must update the user's live GPS location marker on the map with a maximum latency of 1.0 second from receiving the device's signal.
- The live hazard feed must successfully load within 3.0 seconds of the user navigating to the News screen.

5.2 Safety Requirements

- The routing engine shall expire and delete submitted hazardous obstructions and live deductions 72 hours after submission unless confirmed by a subsequent report.
- The system must suppress interactive map interactions and display a motion warning prompt if the device moves faster than 25 km/h during active navigation.
- The system must fail safely with a 100% rejection rate for any route calculation originating or terminating outside Singapore's coordinate boundary, completely preventing unmapped routing in unsupported international border areas.

5.3 Security Requirements

- The backend database must encrypt all stored user passwords using a cryptography hashing algorithm.
- The backend must automatically invalidate a user's active session token after 30 days of inactivity.
- The backend API must sanitize all text inputs against SQL injection and cross-site scripting (XSS) before query execution.

5.4 Software Quality Attributes

- **TESTABILITY:** The routing engine must return an identical route payload when queried with identical Origin, Destination, and preference variables over 100 consecutive requests.
- **ROBUSTNESS:** The app must reload the active navigation screen in 1.5 seconds or less when the user switches back to it from another app.
- **ADAPTABILITY:** The system must apply changes to the confidence score configuration file without requiring a source code recompilation.
- **MAINTAINABILITY:** The routing service must isolate external map provider integrations behind an API adapter interface so changing base map endpoints requires zero alterations to core scoring logic.
- **USABILITY:** At least 95% of first-time users must successfully complete initial profile setup and initiate their first route navigation on their first attempt without external assistance.

5.5 Business Rules

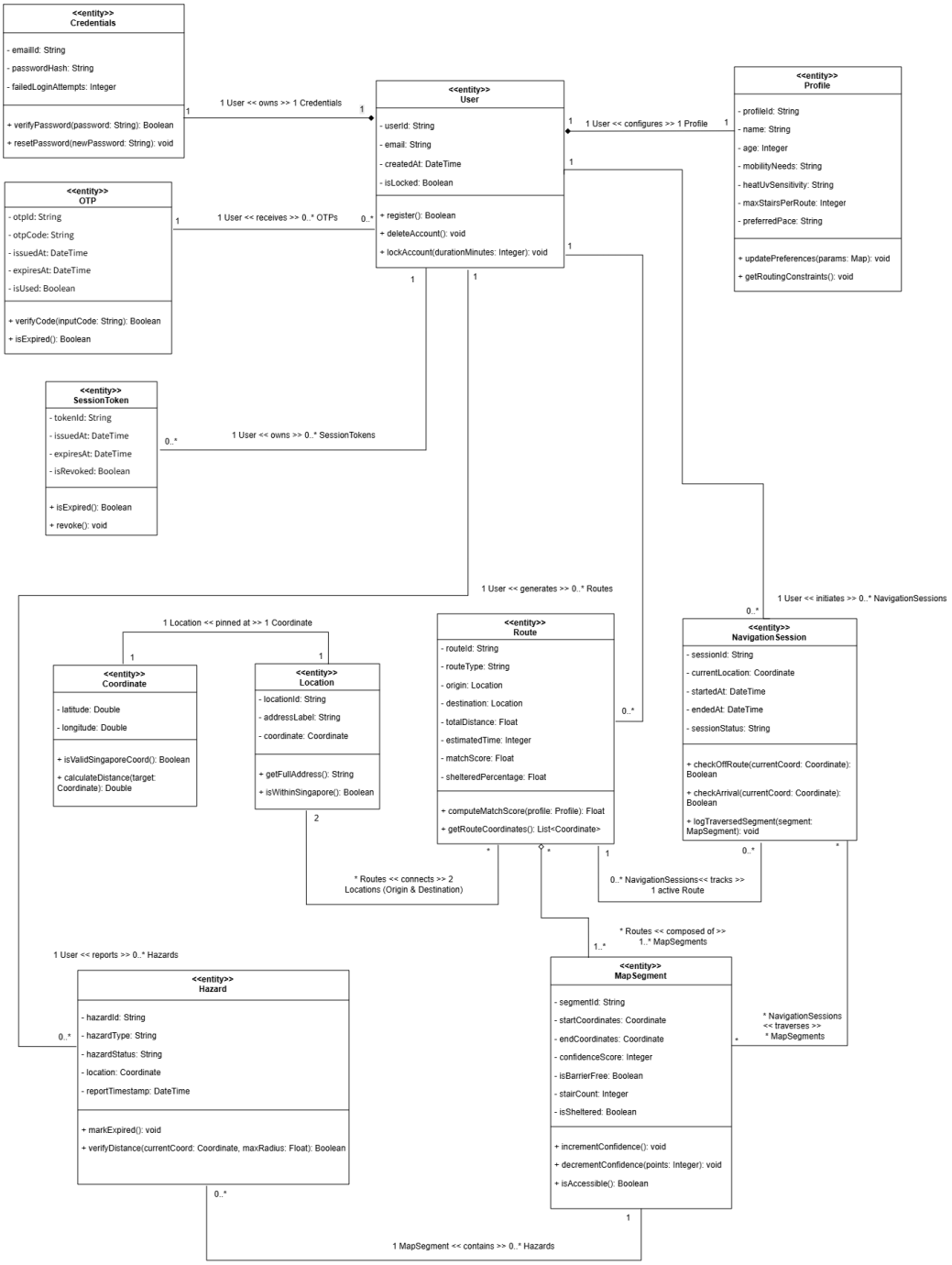
- The system must restore a map segment's confidence score to its default baseline exactly 72 hours after a single "Live Report" deduction.
- The system must grant profile editing privileges exclusively to the user account matching the profile's unique User ID.
- The system must define weights and 0–100 component scores where:
    - S_{eff} is distance efficiency = $\min(100, 100 \times (\text{Distance}_{\text{Direct}} / \text{Distance}_{\text{Route}}))$;
    - $S_{shelter}$ is sheltered distance percentage with weight $w_{shelter}$ (0.20 for High UV, 0.15 for Medium, 0.10 for Low);
    - S_{effort} is $100 - 2 \times \text{stairs_count}$ with weight w_{effort} (0.20 if age ≥ 65 or mobility aid used, else 0.10);
    - S_{conf} is $((\text{avg_confidence} + 20) / 40) \times 100$ with weight $w_{conf} = 0.15$; and $w_{eff} = 0.25$.
    - All weights are normalised so their sum equals 1.0.

- The system must compute each route's match score (0 to 100) using the formula: $Match_Score = G \times (w_{eff} \times S_{eff} + w_{shelter} \times S_{shelter} + w_{effort} \times S_{effort} + w_{conf} \times S_{conf})$, where gate G is 0 if stairs or curbs violate wheelchair constraints, and 1 otherwise.

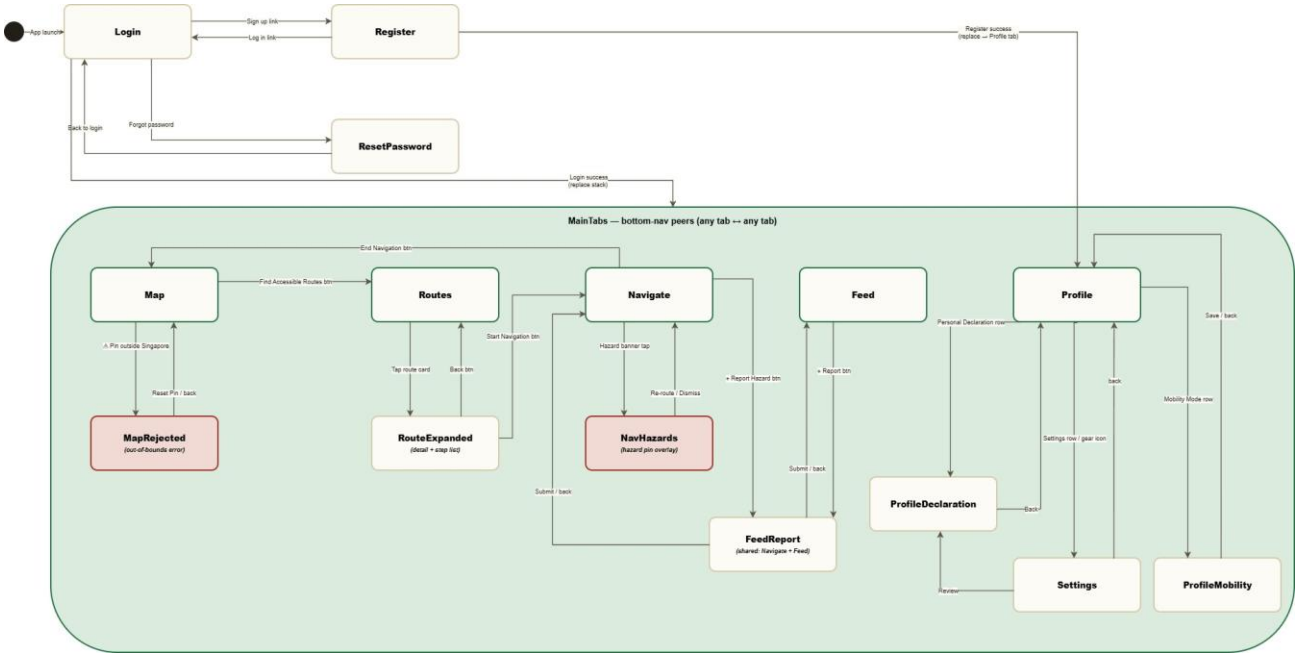
6. Other Requirements

Appendix A: Analysis Models

A.1: Class diagram (Entities)



A.2: Dialog model (Finite-State Model)



A.3: Boundary Classes List

LoginAccountScreen <<boundary>>

- Attributes: emailInput: String, passwordInput: String, otpInput: String, otpInput: String
- Operations: submitRegistration(email: String, password: String): User, submitLogin(email: String, password: String): Boolean, submitPasswordReset(email: String, newPassword: String): void, submitOTP(email: String, otpCode: String): Boolean, resendOTP(email: String): Boolean,

ProfileScreen <<boundary>>

- Attributes: nameInput: String, fullNameInput: String, ageInput: String, mobilityModeInput: String, maxStairsInput: String, maxStairsInput: String, heatSensitivityInput: String, maxDurationInput: String
- Operations: displayProfile(user: User): void, submitPreferences(profile: Profile): void, submitLogout(): void, submitDeleteAccount(): void

SearchMapScreen <<boundary>>

- Attributes: originInput: Location, destinationInput: Location
- Operations: submitRouteSearch(origin: Location, destination: Location): List<Route>, displayLocationError(message: String): void

RouteResultsScreen <<boundary>>

- Attributes: selectedRoute: Route, routesList: List<Route>, selectedSortCriteria: String
- Operations: displayRankedRoutes(routes: List<Route>): void, displayNoRouteMessage(): void, selectRoute(route: Route): void, changeSortCriteria(criteria: String): void, startNavigation(route: Route): void

NavigationScreen <<boundary>>

- Attributes: currentLocation: Coordinate, instruction: String, remainingDistance: Float, estimatedTime: Integer
- Operations: displayInstruction(instruction: String): void, displayTripMetrics(remainingDistance: Float, estimatedTime: Integer): void, endNavigation(), void,displayHazardAlert(hazard: Hazard)

HazardReportScreen <<boundary>>

- Attributes: hazardTypeInput: String, location: Location
- Operations: submitHazardReport(hazardType: String, location: Location): Hazard

HazardFeedScreen <<boundary>>

- Attributes: hazardList: List<Hazard>
- Operations: displayNearbyHazards(hazardList: List<Hazard>): void, navigateToReportHazard(): void

OneMap/MapAPIAdapter <<boundary>>

- Operations: searchAddress(query: String): List<Location>, requestRouteData(origin: Location, destination: Location): List<MapSegment>

A.3: Control Classes List

<<control>>
AuthenticationController

+ registerAccount(email: String, password: String): User
+ login(email: String, password: String): Boolean
+ resetPassword(email: String, newPassword: String): void
+ verifyOTP(user: User, otpCode: String): Boolean
+ logout(sessionToken: SessionToken): void

<<control>>
ProfileController

+ displayUserProfile(userId: String): User
+ createMobilityProfile(userId: String, profile: Profile): Profile
+ updateMobilityProfile(userId: String, profile: Profile): void
+ validateProfile(profile: Profile): Boolean
+ applyPreferenceChanges(profile: Profile): void

<<control>>
LocationSearchController

+ initializeMap(): void
+ searchAddress(query: String): List<Location>
+ validateSingaporeLocation(origin: Location, destination: Location): Boolean
+ setOrigin(origin: Location): void
+ setDestination(destination: Location): void
+ submitRouteSearch(origin: Location, destination: Location): List<Route>

<<control>>
RouteGenerationController

+ generatePersonalizedRoutes(origin: Location, destination: Location, profile: Profile): List<Route>
+ requestRouteSegments(origin: Location, destination: Location): List<MapSegment>
+ generateRouteVariants(segments: List<MapSegment>, profile: Profile): List<Route>
+ filterRestrictedSegments(segments: List<MapSegment>, profile: Profile): List<MapSegment>
+ calculateRouteMetadata(route: Route): void
+ calculateMatchScore(route: Route, profile: Profile): Float
+ rankRoutes(routes: List<Route>): List<Route>

<<control>>
RouteSelectionController

+ selectRoute(route: Route): Route
+ previewRoute(route: Route): void
+ validateRoutePayload(route: Route): Boolean
+ calculatePreviewSummary(route: Route): void
+ startNavigation(route: Route): NavigationSession
+ returnToRouteResults(): void

<<control>>
NavigationController

+ startNavigation(route: Route): NavigationSession
+ updateCurrentLocation(location: Coordinate): void
+ updateInstruction(location: Coordinate, session: NavigationSession): String
+ updateTripMetrics(location: Coordinate, route: Route): void
+ detectRouteDeviation(location: Coordinate, route: Route): Boolean
+ recalculateRoute(currentLocation: Coordinate, destination: Location, routeType: String): Route
+ detectArrival(location: Coordinate, destination: Destination): Boolean
+ completeNavigation(session: NavigationSession): void
+ updateTraversedSegments(session: NavigationSession): void

<<control>>
HazardReportController

+ submitHazardReport(hazardType: String, location: Coordinate, userId: String): Hazard
+ validateHazardType(hazardType: String): Boolean
+ findNearestMapSegment(location: Coordinate): MapSegment
+ createHazard(hazardType: String,location: Coordinate,userId: String,segment: MapSegment): Hazard
+ decreaseConfidenceScore(segment: MapSegment): void
+ publishHazardToFeed(hazard: Hazard): void

<<control>>
HazardFeedController

+ getNearbyHazards(location: Coordinate): List<Hazard>
+ retrieveActiveHazards(): List<Hazard>
+ filterByRadius(hazards: List<Hazard>,location: Coordinate,radiusKm: Float): List<Hazard>
+ filterRecentHazards(hazards: List<Hazard>): List<Hazard>
+ sortByReportTime(hazards: List<Hazard>): List<Hazard>

Appendix B: To Be Determined List

<Collect a numbered list of the TBD (to be determined) references that remain in the SRS so they can be tracked to closure.>

Appendix C: Data Dictionary

Term	Definition	Attribute	Relationships
User	A registered person with an account and credentials logged into the app's database. The user will also have a profile which will have all their preferences according to their needs.	user_id, email, createdAt, isLocked	**One-to-One:** 1 User has exactly 1 set of Credentials **One-to-One:** 1 User has exactly 1 Profile **One-to-Many:** 1 User can generate multiple Routes **One-to-Many:** 1 User can report multiple Hazards **One-to-Many:** 1 User can hold multiple Session Tokens **One-to-Many:** 1 User can be issued multiple OTPs across resend requests.
Credentials	A combination of email and password to authenticate access into the user's account. Credentials are created on registration for the app. Credentials can be reset securely with a link if the user forgets them to regain access.	user_id, email_id, passwordHash, failedLoginAttempts	**One-to-One:** 1 set of Credentials belongs to exactly 1 User
Navigation Session	A session is created when the user confirms a route via the "Proceed" action and remains active until the user arrives at the destination or manually cancels navigation. It tracks the user's real-time position against	Session_id, user_id, route_id, started_at, ended_at, status, segments_traversed	**Many-to-One:** Many Navigation Sessions can belong to 1 User over time **One-to-One:** 1 active Navigation Session

Term	Definition	Attribute	Relationships
	the active route to detect arrival, deviation, and segment traversal for confidence score updates.		references exactly 1 Route. **Many-to-Many:** 1 Navigation Session traverses many Map Segments **Many-to-One:** Many Navigation Sessions can be authorized under 1 Session Token across its active lifetime.
Session Token	A cryptographically generated string issued to a user upon successful login. It represents and authorises an active authenticated session. A token is invalidated on explicit logout, password reset, or after 30 days of inactivity.	token_id, user_id, issued_at, expires_at, last_active_at, is_revoked	**One-to-Many:** 1 Session Token can authorize multiple Navigation Sessions over its lifetime. **Many-to-One:** Many Session Tokens can belong to 1 User
OTP	A system-generated 6-digit numeric code issued to a user's registered email address to verify email ownership during account registration. It is strictly single-use and automatically expires 10 minutes after issuance.	otp_id, user_id, otp_code, issued_at, expires_at, is_used	**Many-to-One:** Many OTPs can be issued to 1 user across multiple resend requests
Profile	Users' data that they have provided to personalise their account. Data includes preferences, personal details and past usage data. Data can be updated by profile when	profileId, name, mobility_needs, age, heat_uv_sensitivity, name, max_stairs_per_route, preferred_pace,	**One-to-One:** 1 Profile belongs to exactly 1 User **One-to-Many:** 1 Profile weights the ranking of multiple Routes

Term	Definition	Attribute	Relationships
	necessary.	max_duration	
Location	Users' location which is obtained via device's GPS location. This can only be obtained when the user provides permission to access this data.	location_id, coordinate, address_label	**One-to-One:** 1 User has at most 1 Location at a time **One-to-One:** 1 Location can be used as the Origin of 1 Route **One-to-One:** 1 Location is pinned at 1 Coordinate **Two-to-Many:** 2 Locations can be connected by many Routes
Coordinate	A set of numbers or letters that pinpoint an exact location on a map, GPS.	latitude, longitude	**One-to-One:** 1 Coordinate represents 1 Location
Origin	The user's starting location which is input to compute a route. Location can be obtained either via autofilling current location or obtaining manual input from the user directly.	coordinate, address_label	**One-to-One:** 1 Route starts from exactly 1 Origin
Destination	The end location for the trip where the user wishes to reach. The location can be obtained via autofilling current location or receiving manual input from the user directly.	destination_id, coordinate, address_label	**One-to-Many:** 1 User can search for multiple Destinations. **One-to-One:** 1 Route computes towards exactly 1 Destination.

Term	Definition	Attribute	Relationships
Route	The computer path from origin to destination which is generated based on the routing algorithm taking into account the total length of the journey and the user's preferences.	user_id, route_id, route_type, total_distance, estimated_time, match_score, sheltered_percentage	**One-to-Many:** 1 User can generate multiple Routes. **Many-to-Two:** Many Routes can connect 2 locations **Many-to-Many:** One Routes can be composed of many Map segments.
Map segments	An individual stretch of walkable path that routes are built from and against which confidence is recorded.	segment_id, start_coordinates, end_coordinates, confidence_score, is_barrier_free, stair_count, is_sheltered	**Many-to-Many:** One Map segment can be shared across Many computed Routes. **One-to-Many:** 1 Map segment can contain multiple reported Hazards. **Many-to-Many:** 1 Map Segment can be traversed across by many Navigation Sessions
Hazard	A hazard is an object or obstruction to a map segment, which could be a part of a computed route for a user, that can cause harm to the user or prevent them from reaching their destination via the route.	hazard_id, hazard_type, hazard_status, location, reported_by_user_id, report_timestamp	**One-to-Many:** 1 User can report multiple Hazards. **One-to-Many:** 1 Map segment can contain multiple Hazards.